

Assignment 3

Python BDD Automation Framework using Selenium and Behave

Submitted by

Adrika Bhakta

B.Tech in Computer Science Engineering (AI & ML)
Institute of Engineering & Management (IEM), Kolkata

Course: Software Testing / Automation Engineering

Topic: Behavior-Driven Development (BDD) with Selenium and Behave

Contents

Overview	2
Project Structure	2
Assignment 1 — Scenario 1: Selenium + Behave Setup and End-to-End Run	3
Day 1 — Environment Setup	3
Day 2–4 — End-to-End Login Scenario (headless-browser simulation)	3
Assignment 2 — Scenario 2: Test-Data-Driven Automation	6
Day 1–4 — Data-Driven API Automation	6
Assignment 3 — Scenario 3: Selenium Page Object Model in Behave	9
Page Object Layer	9
Running the Whole Suite	12
Summary	12

Overview

This assignment implements a complete **Python BDD automation framework** using **Behave** and **Selenium**, covering three linked scenarios:

1. **Scenario 1** – Set up the Selenium + Behave framework and run an end-to-end UI automation scenario (login flow).
2. **Scenario 2** – Extend the framework for **test-data-driven automation** (API automation driven by external JSON test data).
3. **Scenario 3** – Refactor the UI tests using the **Page Object Model (POM)** for a cleaner, maintainable Behave/Gherkin suite.

All three scenarios share one project so that step definitions, configuration, and reporting are reused rather than duplicated. The project runs in **PyCharm** (Community edition is sufficient — no paid features are used).

Project Structure

```
selenium_behave_framework/
|-- behave.ini
|-- requirements.txt
|-- testdata/
|   |-- users.json
|-- features/
|   |-- environment.py
|   |-- login.feature                # Scenario 1
|   |-- api_data_driven.feature     # Scenario 2
|   |-- search_pom.feature          # Scenario 3
|   |-- steps/
|       |-- login_steps.py
|       |-- api_steps.py
|       |-- search_steps.py
|-- pages/
|   |-- base_page.py
|   |-- login_page.py
|   |-- search_page.py
```

Assignment 1 — Scenario 1: Selenium + Behave Setup and End-to-End Run

Objective: Set up the Python BDD framework (Behave + Selenium), debug it in PyCharm, and run an end-to-end UI scenario.

Day 1 — Environment Setup

Note on the simulation step: the original task sheet for this slot references setting up **QEMU** and **Eclipse + GDB** for an embedded-hardware simulation. That tooling belongs to an embedded-systems lab, not a Python/Selenium/Behave web-automation lab, so it is not applicable here. The equivalent “simulated environment instead of actual hardware” for a **web** BDD framework is a **headless browser** (Chrome/Firefox in headless mode), which is what Day 2-4 below actually exercises instead of a real, manually-operated browser window.

Install dependencies:

```
pip install behave selenium webdriver-manager requests
```

requirements.txt:

```
behave==1.2.6
selenium==4.23.1
webdriver-manager==4.0.2
requests==2.32.3
```

PyCharm setup:

1. File > New Project → select the project folder above, choose a virtualenv interpreter.
2. Install the **Gherkin** and **BDD** plugins (Settings > Plugins) so .feature files get syntax highlighting and step navigation.
3. Mark features/steps as a **Sources Root** so imports resolve cleanly.
4. Add a **Behave** run configuration (Run > Edit Configurations > +) pointing at the features directory, for one-click debugging with breakpoints in step files.

behave.ini (project-wide config):

```
[behave]
paths = features
default_tags = ~@skip
junit = true
junit_directory = reports
```

Day 2–4 — End-to-End Login Scenario (headless-browser simulation)

features/environment.py — shared browser lifecycle for every scenario in the project:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
```

```

def before_all(context):
    """Runs once before the whole test run."""
    context.base_url = "https://the-internet.herokuapp.com"

def before_scenario(context, scenario):
    """Runs before every scenario: start a fresh, headless Chrome session."""
    options = Options()
    options.add_argument("--headless=new") # simulated environment, no real display
    options.add_argument("--window-size=1280,900")
    options.add_argument("--disable-gpu")
    service = Service(ChromeDriverManager().install())
    context.driver = webdriver.Chrome(service=service, options=options)
    context.driver.implicitly_wait(5)

def after_scenario(context, scenario):
    """Runs after every scenario: close the browser and log failures."""
    if scenario.status == "failed":
        context.driver.save_screenshot(f"reports/{scenario.name}.png")
        context.driver.quit()

```

features/login.feature:

Feature: Login

As a registered user

I want to log in with valid or invalid credentials

So that I can access, or be correctly denied, the secure area

Scenario: Successful login with valid credentials

Given I open the login page

When I log in with username "tomsmith" and password "SuperSecretPassword!"

Then I should see the secure area message

Scenario: Failed login with invalid credentials

Given I open the login page

When I log in with username "tomsmith" and password "wrongpassword"

Then I should see an invalid credentials error

features/steps/login_steps.py:

```

from behave import given, when, then
from selenium.webdriver.common.by import By

```

```

LOGIN_URL = "/login"

```

```

@given("I open the login page")
def step_open_login(context):
    context.driver.get(context.base_url + LOGIN_URL)

@when('I log in with username "{username}" and password "{password}"')
def step_login(context, username, password):
    context.driver.find_element(By.ID, "username").send_keys(username)
    context.driver.find_element(By.ID, "password").send_keys(password)
    context.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

@then("I should see the secure area message")
def step_secure_area(context):
    message = context.driver.find_element(By.ID, "flash").text
    assert "You logged into a secure area" in message, f"Unexpected message: {message}"

@then("I should see an invalid credentials error")
def step_invalid_login(context):
    message = context.driver.find_element(By.ID, "flash").text
    assert "invalid" in message.lower(), f"Unexpected message: {message}"

```

Run and debug from PyCharm's Behave run configuration, or from the terminal:

```

behave features/login.feature -f pretty

```

Expected result: both scenarios pass — one confirms the success message, the other confirms the error message. Breakpoints can be set directly inside `login_steps.py` in PyCharm's debugger to inspect `context.driver` at any step.

Assignment 2 — Scenario 2: Test-Data-Driven Automation

Objective: Drive Behave scenarios from external test data (JSON) for API automation, so the same steps validate many input/output combinations without duplicating Gherkin.

Day 1–4 — Data-Driven API Automation

testdata/users.json — external, editable test data:

```
[
  {"endpoint": "/posts/1", "expected_status": 200, "expected_key": "title"},
  {"endpoint": "/posts/9999", "expected_status": 404, "expected_key": null},
  {"endpoint": "/users/1", "expected_status": 200, "expected_key": "email"}
]
```

features/api_data_driven.feature — a **Scenario Outline** turns each row of an Examples table into its own scenario run, and can also be paired with data pulled straight from users.json:

Feature: Data-driven API automation

As an API consumer

I want each endpoint validated against expected status codes

So that regressions are caught automatically

Scenario Outline: Validate API response for <endpoint>

Given the API base URL is set

When I send a GET request to "<endpoint>"

Then the response status should be <status>

Examples: Endpoints from JSON test data

endpoint	status	
/posts/1	200	
/posts/9999	404	
/users/1	200	

Scenario: Validate every record in the external JSON data file

Given the API base URL is set

When I run all requests from "testdata/users.json"

Then every response should match its expected status and key

features/steps/api_steps.py:

```
import json
import requests
from behave import given, when, then

BASE_URL = "https://jsonplaceholder.typicode.com"

@given("the API base URL is set")
def step_set_base_url(context):
```

```

context.api_base_url = BASE_URL

@when('I send a GET request to "{endpoint}"')
def step_send_get(context, endpoint):
    context.response = requests.get(context.api_base_url + endpoint, timeout=10)

@then("the response status should be {status:d}")
def step_check_status(context, status):
    actual = context.response.status_code
    assert actual == status, f"Expected {status}, got {actual}"

@when('I run all requests from "{data_file}"')
def step_run_all_from_file(context, data_file):
    with open(data_file, encoding="utf-8") as f:
        context.test_records = json.load(f)

    context.results = []
    for record in context.test_records:
        response = requests.get(context.api_base_url + record["endpoint"], timeout=10)
        context.results.append((record, response))

@then("every response should match its expected status and key")
def step_verify_all(context):
    failures = []
    for record, response in context.results:
        if response.status_code != record["expected_status"]:
            failures.append(
                f"{record['endpoint']}: expected status "
                f"{record['expected_status']}, got {response.status_code}"
            )
        continue
        if record["expected_key"] and record["expected_key"] not in response.json():
            failures.append(
                f"{record['endpoint']}: missing key '{record['expected_key']}'"
            )

    assert not failures, "Data-driven check failures:\n" + "\n".join(failures)

```

Run just this feature:

```

behave features/api_data_driven.feature -f pretty

```

Expected result: the Scenario Outline runs three times, once per Examples row, and the JSON-driven scenario runs every record in `users.json` in a single loop, reporting all failures to-

gether instead of stopping at the first one.

Assignment 3 — Scenario 3: Selenium Page Object Model in Behave

Objective: Debug and optimize a web-application automation suite by introducing the **Page Object Model (POM)** underneath the existing Behave / Gherkin layer, and drive it with a data-table (test-data-driven) scenario.

Page Object Layer

features/pages/base_page.py:

```
class BasePage:
    """Common behaviour shared by every page object."""

    def __init__(self, driver):
        self.driver = driver

    def open(self, path, base_url):
        self.driver.get(base_url + path)

    def text_of(self, locator):
        return self.driver.find_element(*locator).text
```

features/pages/search_page.py:

```
from selenium.webdriver.common.by import By
from features.pages.base_page import BasePage

class SearchPage(BasePage):
    PATH = "/"

    SEARCH_INPUT = (By.NAME, "q")
    RESULT_STATS = (By.ID, "result-stats")
    RESULT_LINKS = (By.CSS_SELECTOR, "h3")

    def open_search_engine(self, base_url):
        self.open(self.PATH, base_url)

    def search_for(self, term):
        box = self.driver.find_element(*self.SEARCH_INPUT)
        box.send_keys(term)
        box.submit()

    def get_result_count_text(self):
        return self.text_of(self.RESULT_STATS)

    def get_result_titles(self):
        return [el.text for el in self.driver.find_elements(*self.RESULT_LINKS)]
```

Keeping locators and low-level Selenium calls inside the page object — rather than in the step file — is what let this suite be *debugged and optimized*: when the site's markup changes, only `search_page.py` needs an edit, not every step definition that touches that page.

`features/search_pom.feature` — data-table-driven scenario:

```
Feature: Search using the Page Object Model
  As a user of the search engine
  I want to search for several terms
  So that I get relevant, non-empty results for each one

  Scenario: Search returns results for multiple terms
    Given I open the search page
    When I search for each term below
      | term          |
      | Behave BDD    |
      | Selenium Python |
      | Page Object Model |
    Then every search should return at least one result
```

`features/steps/search_steps.py`:

```
from behave import given, when, then
from features.pages.search_page import SearchPage

@given("I open the search page")
def step_open_search(context):
    context.search_page = SearchPage(context.driver)
    context.search_page.open_search_engine(context.base_url)

@when("I search for each term below")
def step_search_each_term(context):
    context.search_outcomes = []
    for row in context.table:
        term = row["term"]
        context.search_page.search_for(term)
        titles = context.search_page.get_result_titles()
        context.search_outcomes.append((term, titles))
        context.search_page.open_search_engine(context.base_url) # reset for next term

@then("every search should return at least one result")
def step_verify_results(context):
    empty = [term for term, titles in context.search_outcomes if not titles]
    assert not empty, f"No results returned for: {' '.join(empty)}"
```

`context.base_url` for this feature should point at a real search engine (e.g. <https://www.bing.com/search>) rather than `the-internet.herokuapp.com` used in Assignment 1 — override it in

`before_scenario` with a tag, e.g. `@search` on this feature, if it lives alongside the other two.

Run and debug:

```
behave features/search_pom.feature -f pretty --no-capture
```

Expected result: the scenario iterates over every row of the Gherkin data table, reuses one `SearchPage` object per search, and fails with a single clear assertion listing exactly which search term(s) returned nothing — making the suite both **data-driven** and easy to **debug**, since a failure points at one method in `search_page.py` instead of a tangle of inline Selenium calls.

Running the Whole Suite

From the project root, all three feature files run together with a single JUnit + pretty report, matching the `behave.ini` config from Assignment 1:

```
behave

reports/
|-- TESTS-login.feature.xml
|-- TESTS-api_data_driven.feature.xml
`-- TESTS-search_pom.feature.xml
```

Summary

Scenario	Focus	Key technique
1	Framework setup + E2E login	Behave <code>environment.py</code> , headless Selenium
2	Data-driven automation	<code>Scenario Outline</code> + external JSON test data
3	Page Object Model	Locators/actions moved out of steps into page classes